

FloodRoute HCMC

Tìm kiếm tuyến đường giao hàng trong điều kiện kẹt xe và ngập nước

Nhóm FloodRoute

Lưu Huy Minh Quang Vũ Lê Trọng Văn

Nguyễn Duy Khang Ngô Quang Thắng / Tartz User

Nhập môn Trí tuệ Nhân tạo – ĐH Khoa học Tự nhiên, ĐHQG-HCM

Giảng viên: Bùi Duy Đăng Bùi Tiến Lên Võ Nhật Tân



Thành viên và phân bổ điểm

Thành viên	Điểm
Lưu Huy Minh Quang	35%
Vũ Lê Trọng Văn	25%
Nguyễn Duy Khang	20%
Ngô Quang Thắng / Tartz User	20%
Tổng	100%

Mục tiêu: minh họa thuật toán tìm kiếm trên một bài toán giao thông thực tế

Bối cảnh giao thông tại TP. Thủ Đức

Giao thông đô thị

- ▶ Mạng đường có nhiều đoạn một chiều và hẻm kết nối.
- ▶ Giờ cao điểm: 07:00–08:30 và 16:30–18:30.
- ▶ Kẹt xe làm tăng thời gian di chuyển trên một số tuyến huyết mạch.

Ngập cục bộ

- ▶ Khu vực Chợ Thủ Đức có các điểm trũng và thoát nước quá tải.
- ▶ Cạnh ngập có thể làm tuyến ngắn nhất trở nên không phù hợp.
- ▶ Mục tiêu là tìm tuyến ngắn nhất nhưng vẫn khả thi trong scenario.

Bài toán đặt ra

Đầu vào

Điểm bắt đầu, điểm đích, đồ thị có hướng và một scenario môi trường.

Đầu ra

Một route gồm chuỗi node/edge, khoảng cách, ETA, total cost và trace tìm kiếm.

Ưu tiên của nhóm

Trong điều kiện bình thường, ưu tiên tuyến đường ngắn nhất khả thi. Khi có cản trở, chọn tuyến có composite cost thấp nhất trong các tuyến còn khả thi.

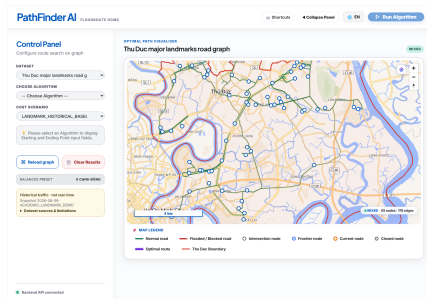
- ❶ Mô hình hóa mạng đường dưới dạng graph có hướng bất biến.
- ❷ Cài đặt và so sánh sáu thuật toán tìm kiếm.
- ❸ Đưa kẹt xe, ngập nước và closure vào scenario.
- ❹ Giải thích route bằng cost breakdown, ETA và tuyến thay thế.
- ❺ Hỗ trợ bài toán nhiều điểm giao hàng bằng Held–Karp và Nearest Neighbor.

65

Landmark nodes

283

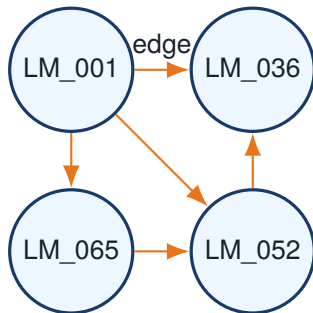
Directed aggregate edges



Landmark map và trạng thái các edge.

- Dataset: thu_duc_landmarks_v1.0.4.
- Giữ polyline từ UTraffic.
- Traffic/flood: historical, simulated hoặc assumption.

Mô hình graph



- ▶ Node: landmark và tọa độ địa lý.
- ▶ Edge: hướng đi, khoảng cách, thời gian, traffic, flood và trạng thái đóng.
- ▶ Graph không bị mutate trong quá trình tìm kiếm; tie-break deterministic.

Pipeline dữ liệu

RAW
snapshot

INTERIM
lọc vùng

PROCESSED
release

SUBMISSION
bundle

Nguyên tắc dữ liệu

Raw data bất biến; release có manifest, validation report và SHA-256 checksum.

Các trường dữ liệu quan trọng

Trường	Ý nghĩa	Nhãn
distance_m	Chiều dài vật lý của aggregate edge	DERIVED
free_flow_time_min	Thời gian khi đường thông	SOURCE/DERIVED
traffic_penalty_min	Độ trễ do kẹt xe	SIMULATED
flood_risk_0_1	Mức rủi ro ngập chuẩn hóa	ASSUMPTION
is_closed	Edge có bị loại khỏi tìm kiếm không	SCENARIO

Free-flow time

$$t(e) = \frac{distance_km(e)}{speed_kmh(e)} \times 60; \text{ connector landmark dùng giả định } 20 \text{ km/h.}$$

Hàm chi phí 4 thành phần

$$C_s(e) = w_d D(e) + w_t T_{ff}(e) + w_c P_{traffic,s}(e) + w_f R_{flood,s}(e)$$

Distance

Khoảng cách

Free-flow

Thời gian cơ sở

Congestion

Độ trễ kẹt xe

Flood risk

Rủi ro ngập

Closure

Edge đóng là hard constraint: không được xuất hiện trong neighbors hoặc route, bất kể weight.

Cost profiles

Profile	Distance	Time	Congestion	Flood
BALANCED	0.25	0.30	0.20	0.25
PEAK_TRAFFIC	0.15	0.25	0.45	0.15
RAIN_SAFE	0.10	0.25	0.15	0.50

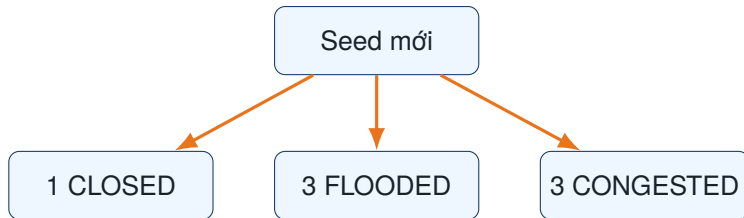
Các weight là hệ số ưu tiên của prototype, không phải phần trăm đóng góp vì các thành phần khác đơn vị và scale.

Bốn scenario trên frontend

Scenario	Cách hoạt động
NORMAL	Không có cản trở; baseline cho tuyến ngắn nhất khả thi.
CONGESTION	57/283 edge có traffic penalty, xấp xỉ 20%.
FLOOD	57/283 edge có flood risk, xấp xỉ 20%.
RANDOM	Mỗi lần nhấn sinh seed mới và chọn đồng thời closure, flood, congestion.

SIMULATED / ASSUMPTION – không phải dữ liệu real-time

Random scenario: cả kẹt và ngập



Mỗi lần regenerate, tập edge và trạng thái trên map được thay mới.

Hard closure và detour

- ▶ `closed_edge_ids` và `edge_overrides.is_closed` đều được Graph contract xử lý thống nhất.
- ▶ Tất cả thuật toán chỉ nhìn thấy edge mở trong `'neighbors()'`.
- ▶ Nếu không còn tuyến, API trả `NO_ROUTE`.
- ▶ Closure áp dụng theo landmark edge; không lan sang aggregate edge khác chỉ vì hình học chồng lấn.

Kết quả mong muốn

Không thuật toán nào được phép đi xuyên qua cạnh đã đóng.

Sáu thuật toán tìm kiếm

Nhóm	Thuật toán	Đặc điểm
Không trọng số	BFS	Ít số chặng nhất; không tối ưu cost tổng quát
Không trọng số	DFS	Khám phá sâu; không bảo đảm tối ưu
Có trọng số	UCS/Dijkstra	Tối ưu composite cost
Có trọng số	A*	UCS + heuristic lower-bound
Heuristic	Greedy Best-First	Nhanh; không bảo đảm tối ưu
Mở rộng	Bidirectional	Tìm từ hai hướng; cần điều kiện dừng phù hợp

BFS

- ▶ Queue FIFO.
- ▶ Mở rộng theo độ sâu.
- ▶ Tối ưu số hop khi mọi edge có cùng cost.

DFS

- ▶ Stack LIFO.
- ▶ Đi sâu theo một nhánh trước.
- ▶ Dùng expanded để tránh chu trình.

Cả hai vẫn tuân thủ Graph contract và hard closure.

Nguyên lý

Mở rộng node có $g(n)$ nhỏ nhất bằng min-heap.

$$g(v) = \min_{u \rightarrow v} \{g(u) + C_s(u, v)\}$$

- ▶ Chi phí cạnh không âm $\Rightarrow$ UCS tìm được C^* nhỏ nhất.
- ▶ Được dùng làm chuẩn đối chiếu tính đúng đắn của A^* .
- ▶ Không dùng heuristic nên thường mở rộng nhiều node hơn A^* .

$$f(n) = g(n) + h_s(n)$$

- ▶ $g(n)$: cost đã tích lũy từ start.
- ▶ $h_s(n)$: lower bound từ node hiện tại đến goal.
- ▶ Scenario được freeze trong một lần search.
- ▶ A* trả route tối ưu theo composite cost khi heuristic admissible và consistent.

$$\rho_s = \min_{e \text{ open}} \frac{C_s(e)}{D_{geo}(e)}, \quad h_s(n) = \rho_s D_{geo}(n, goal)$$

- ▶ $C_s(e)$ đã bao gồm distance, time, congestion và flood risk.
- ▶ Edge đóng và cạnh có mẫu địa lý gần 0 không tham gia tính ρ_s .
- ▶ Heuristic không dư dữ kiện; nó là lower-bound cho đúng cost hiện tại.

Bảo đảm đúng đắn

Admissibility

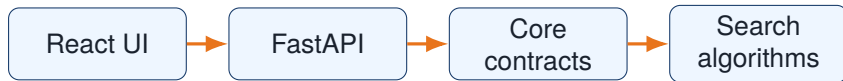
$h_s(n)$ không vượt quá cost thật của bất kỳ đường đi nào từ n đến goal.

Consistency

$h_s(u) \leq C_s(u, v) + h_s(v)$ với mọi edge mở.

Kết luận

UCS và A* tối ưu composite cost; route ngắn nhất được ưu tiên thông qua cost cơ sở và cách diễn giải kết quả.

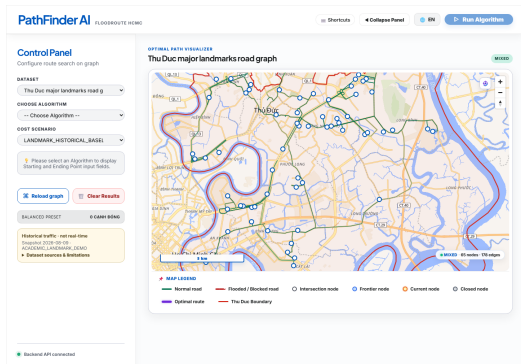


Graph và ScenarioCostEngine độc lập với FastAPI; thuật toán không mutate graph.

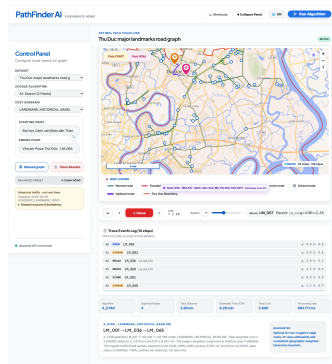
Luồng chạy trên frontend

- 1 Chọn start, goal, algorithm và scenario.
- 2 Gọi API search với graph/scenario hiện tại.
- 3 Hiển thị route chính, trace frontier/current/closed và metrics.
- 4 Nếu scenario có cản trở, gọi endpoint alternatives cùng algorithm.
- 5 Người dùng chọn route để map chuyển highlight sang tuyến tương ứng.

Minh họa giao diện: bản đồ và route

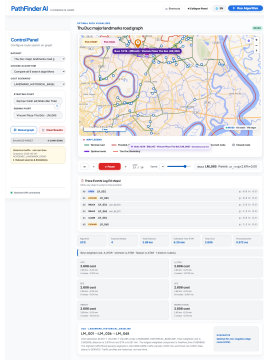


Bản đồ landmark và trạng thái edge.

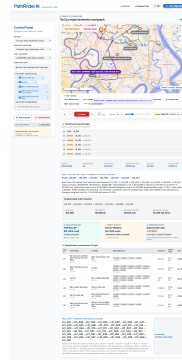


Route và trace của A*.

Minh họa giao diện: so sánh và tour



So sánh kết quả các thuật toán.



Tối ưu tour nhiều điểm giao hàng.

Các tuyến thay thế

- ▶ Dùng chính thuật toán người dùng đã chọn, không trộn kết quả của thuật toán khác.
- ▶ Tạm loại một edge trên route chính để tìm detour mới.
- ▶ Trả tối đa hai route thay thế khác nhau.
- ▶ Mỗi thẻ hiển thị distance, ETA và total cost.
- ▶ Thẻ có cost thấp nhất được đánh dấu tối ưu trong scenario hiện tại.

Màu	Trạng thái	Ý nghĩa
Xanh lá	Open	Đường lưu thông
Vàng	Congested	Đường bị kẹt
Xanh dương	Flooded	Đường bị ngập
Đỏ nét đứt	Closed	Đường bị đóng
Tím	Selected	Tuyến đang được chọn

Legend song ngữ Việt/Anh được đặt trực tiếp trên map.

Bài toán nhiều điểm giao hàng

Pairwise matrix

Tính cost và thời gian ngắn nhất giữa depot và các điểm giao.

Held–Karp DP

Nghiệm chính xác cho số stop nhỏ. Độ phức tạp tăng theo cấp số mũ.

Nearest Neighbor

Heuristic nhanh: mỗi bước chọn điểm chưa thăm có pairwise cost nhỏ nhất.

- ▶ Data validator: kiểm tra manifest, checksum, schema và số node/edge.
- ▶ Backend tests: contract, cost, closure, random scenario, search.
- ▶ Frontend tests: scenario selector, map status, route selection.
- ▶ A*/UCS lower-bound: so sánh cost và số node khám phá.
- ▶ Build LaTeX: biên dịch hai lượt và render kiểm tra bố cục.

Kết quả và cách diễn giải

Điều được chứng minh

Trong các case kiểm thử, A* trả cùng composite cost với UCS khi dùng cùng graph và scenario.

Điều được quan sát

Khi kẹt/ngập làm tăng cost hoặc closure loại cạnh, route có thể đổi sang detour dài hơn nhưng khả thi hơn.

Cách gọi chính xác

“Tuyến tối ưu theo composite cost”, không khẳng định là khoảng cách ngắn nhất tuyệt đối trong mọi scenario.

- ▶ Traffic là historical snapshot, chưa phải live telemetry.
- ▶ Flood là danh mục/risk giả lập, chưa có hydraulic dynamic model.
- ▶ Weight là hệ số prototype; các đơn vị chưa được chuẩn hóa thành tiền hoặc thời gian duy nhất.
- ▶ Aggregate edge giữ topology landmark, không phải toàn bộ mạng road segment.
- ▶ Random scenario có seed mới mỗi lần nên cần lưu seed nếu muốn tái lập chính xác.

Các trường hợp có thể làm kết quả sai

- ▶ **Snap sai start/goal:** địa chỉ giữa hai landmark bị gán vào node gần nhất nên route bắt đầu/kết thúc lệch vị trí.
- ▶ **Sai hướng cạnh:** graph chỉ có LM_001 → LM_036; tìm chiều ngược có thể trả NO_ROUTE dù người dùng nghĩ đường hai chiều.
- ▶ **Closure không theo hình học:** đóng một landmark edge không tự đóng aggregate edge khác đang chồng lấn cùng đoạn đường.
- ▶ **Scenario bị stale:** RANDOM đổi seed sau khi search làm màu edge, cost và route không còn cùng một trạng thái.
- ▶ **Cost khác mục tiêu:** A*/UCS tối ưu composite cost, không đảm bảo ngắn nhất theo mét nếu flood/congestion weight cao.
- ▶ **Graph thừa hoặc dữ liệu lỗi:** closure có thể dẫn đến NO_ROUTE; tọa độ đảo lat/lon hoặc polyline lỗi cũng làm khoảng cách sai.

Cách diễn giải

Kết quả có thể đúng theo graph và contract, nhưng vẫn không đúng với thực tế nếu dữ liệu, scenario hoặc cost model không đại diện đủ.

Định hướng phát triển²

- ➊ Kết nối traffic và flood real-time.
- ➋ Hiệu chỉnh tốc độ theo dữ liệu đo thực tế.
- ➌ Tách rõ chế độ tối ưu khoảng cách và composite cost.
- ➍ Hỗ trợ custom weights có validation.
- ➎ Mở rộng nhiều xe, capacity và time windows.

- ▶ FloodRoute mô hình hóa bài toán giao thông bằng graph 65 node/283 directed edge.
- ▶ Mục tiêu chính là ưu tiên tuyến đường ngắn nhất khả thi.
- ▶ Kẹt xe, ngập nước và closure được thể hiện minh bạch trên map và trong cost.
- ▶ UCS/A* cung cấp nghiệm tối ưu theo composite cost; các thuật toán khác giúp so sánh hành vi tìm kiếm.
- ▶ Frontend cho phép quan sát route, trace, scenario và tuyến thay thế trong cùng một giao diện.

Cảm ơn thầy cô và các bạn đã lắng nghe!